

1. Consider a variant of the matrix-chain multiplication problem in which the goal is to parenthesize the sequence of matrices so as to maximize, rather than minimize, the number of scalar multiplications.

- (a) Does this problem exhibit optimal substructure?
- (b) Find an optimal parenthesization of matrix-chain product whose sequence of dimensions is $\langle 10, 3, 12, 5, 50 \rangle$.
- (c) Give an algorithm to solve this problem.

(a) - 2 分

評分標準：

答「是」：1 分

理由充分：1 分

參考解答：

對於矩陣鏈 $A_i \cdots A_j$ 而言，若要在 k 處切分並使得乘法次數最大化，則左子鏈 $A_i \cdots A_k$ 與右子鏈 $A_{k+1} \cdots A_j$ 也必須分別取其各自的「最大化」乘法次數。因為子問題的解相互獨立（左右子鏈的切法不會互相影響）且可以組合（矩陣鏈的答案可以由左右子鏈的答案組合得到），所以滿足最佳子結構。

(b) - 4 分

評分標準：

過程：2 分 (筆墨分 1 分)

正確的括號對：1 分

正確的數值：1 分

參考解答：

設 4 個矩陣為 $A_1(10 \times 3)$, $A_2(3 \times 12)$, $A_3(12 \times 5)$, $A_4(5 \times 50)$ 。

計算乘法數公式： $A_i(p_{i-1} \times p_i)$ 與 $A_j(p_{j-1} \times p_j)$ 相乘代價為 $p_{i-1} \times p_i \times p_j$ 。

令 $T_{i,j}$ 是 $A_i A_{i+1} \cdots A_j$ 的最大純量乘法次數，根據最佳子結構可以知道：

$$T_{i,j} = \max_{i \leq k < j} (T_{i,k} + T_{k+1,j} + p_{i-1} * p_k * p_j)$$

$T_{i,j}$	$i = 1$	2	3	4
$j = 1$	0	X	X	X
2	360 ($k = 1$)	0	X	X
3	960 ($k = 2$)	180 ($k = 2$)	0	X
4	9360 ($k = 2$)	4800 ($k = 2$)	3000 ($k = 3$)	0

最佳括號方式為： $((A_1 A_2)(A_3 A_4))$ ，最大乘法次數為 9360。

(c) - 4 分

評分標準：

完全正確：4 分 (筆墨分視完整性給予 1~2 分)

參考解答：

使用 (b) 找出的轉移式設計動態規劃演算法：

1. 建立二維表格 $T[1..N][1..N]$ ，初始化對角線為 0。
2. 對鏈長度 $l = 2 \sim N$ 進行迭代。
3. 對於所有長度 l 的矩陣鏈 $A[i..i+l-1]$ 遍歷所有分割點 k 並計算

$$T_{i,j} = \max_{i \leq k < j} (T_{i,k} + T_{k+1,j} + p_{i-1} * p_k * p_j)$$

```
solve(p, N){
    int T[N+1][N+1], S[N+1][N+1]; // S 用來紀錄最佳分割點
    for(int i = 1; i <= N; i++){ // O(N)
        for(int j = 1; j <= N; j++){ // O(N)
            T[i][j] = 0; // O(1)
        }
    }
    for(int l = 2; l <= N; l++){ // O(N)
        for(int i = 1, j = l; j <= N; i++, j++){ // O(N)
            for(int k = i; k < j; k++){ // O(N)
                int q = T[i][k] + T[k+1][j] + p[i-1]*p[k]*p[j]; // O(1)
                if(q > T[i][j]){ // O(1)
                    T[i][j] = q;
                    S[i][j] = k;
                }
            }
        }
    }
    return T, S;
}
```

時間複雜度 $O(N^3)$